WhatsApp

Security of End-To-End Encrypted Backups

WhatsApp Security Whitepaper

Version 2 Updated May 1, 2026

Version 1 Originally published September 10, 2021

Contents

Introduction

This whitepaper provides a technical explanation of WhatsApp's implementation of end-to-end encrypted backups, used by WhatsApp and Facebook Messenger. For additional context and information on WhatsApp's end-to-end message encryption, please visit WhatsApp's website at www.whatsapp.com/security.

Since 2016, all personal messages, calls, video chats and media sent on WhatsApp have been end-to-end encrypted. This means that no one except the user, not even WhatsApp, can access them.

The content of message chats is valuable to WhatsApp users and WhatsApp offers an in-app backup feature to protect the content in the event a user's device is lost or stolen; and to enable the transfer of their chat history to a new device.

WhatsApp's backup management relies on mobile device cloud partners, such as Apple and Google, to store backups of the WhatsApp data (chat messages, photos, etc.) in Apple iCloud or Google Drive. Prior to the introduction of end-to-end encrypted backups, backups stored on Apple iCloud and Google Drive were not protected by WhatsApp's end-to-end encryption. Now we are offering the ability to secure your backups with end-to-end encryption before they are uploaded to these cloud services.

With the introduction of end-to-end encrypted backups, WhatsApp has created an HSM (Hardware Security Module) based Backup Key Vault to securely store per-user encryption keys for user backups in tamper-resistant storage, thus ensuring stronger security of users' message history.

With end-to-end encrypted backups enabled, before storing backups in the cloud, the client encrypts the chat messages and all the messaging data (i.e. text, photos, videos, etc) that is being backed up using a random key that's generated on the user's device.

The key to encrypt the backup is secured with a user-provided password. The password is unknown to WhatsApp, the user's mobile device cloud partners, or any third party. An encrypted version of the key is stored in the HSM Backup Key Vault to allow the user to recover the key in the event the device is lost or stolen. The HSM Backup Key Vault is responsible for enforcing password verification attempts and rendering the key permanently inaccessible after a certain number of unsuccessful attempts to access it. These security measures provide protection against brute force attempts to retrieve the key.

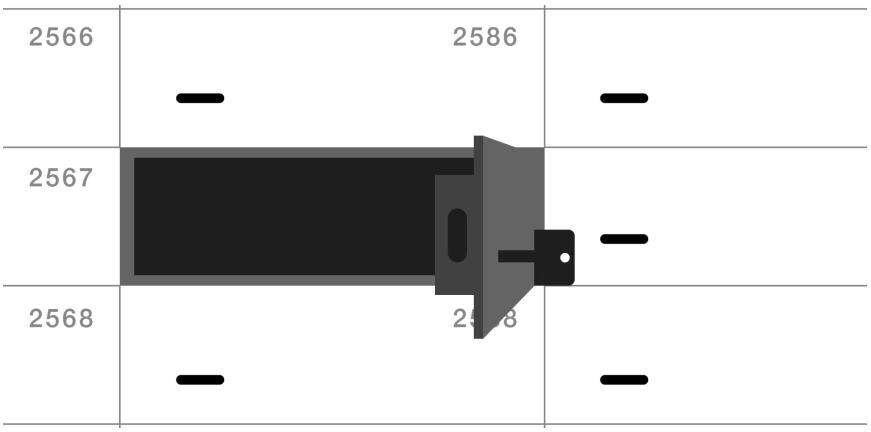
Additionally, the users have a choice to use a 64-digit encryption key instead of a password, which would require them to remember the encryption key themselves or store it manually as in this case the key is not sent to the HSM Backup Key Vault.

Currently, end-to-end encrypted backups are only supported on a user's primary device. In addition, we recommend that users who opt in to end-to-end encrypted backups also deselect WhatsApp from the apps that are included in their

device-level backups. We will inform users of the need to do this when they set up their end-to-end encrypted backup in WhatsApp.

HSM-based Backup Key Vault as a safe deposit box

WhatsApp's HSM-based Backup Key Vault can be compared to safe deposit boxes that are often offered by conventional banks.



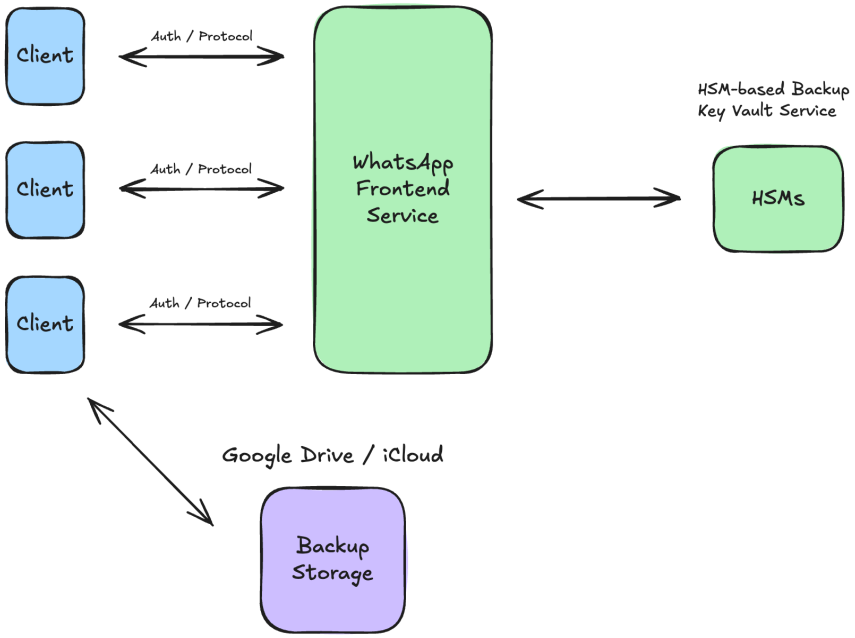
In a conventional bank, a user is given a key to their box with an assurance that even bank employees are not able to access the contents of the box.

WhatsApp's HSM-based Backup Key Vault provides a similar assurance, which is that only the user and nobody else, including Apple, Google, Facebook, or WhatsApp, is able to access a user's backed-up messages.

Such assurance enhances the security and privacy of the user's data. Such assurance is also similar to end-to-end encryption used in WhatsApp messaging: only the users that are engaged in a message exchange are able to access the contents of the messages, while WhatsApp infrastructure does not have access.

System overview

Clients connect to WhatsApp via the WhatsApp front-end service. WhatsApp front-end service handles client connections, client-server authentication, as well as implements the protocol for uploading and downloading of encrypted backup keys.



Behind the front end is the HSM-based Backup Key Vault service. The purpose of that service is to provide highly-available secure storage of encrypted keys for end-to-end encrypted backups. The HSM-based Backup Key Vault service is geographically-distributed across multiple datacenters, allowing it to continue operating in the event of a datacenter outage.

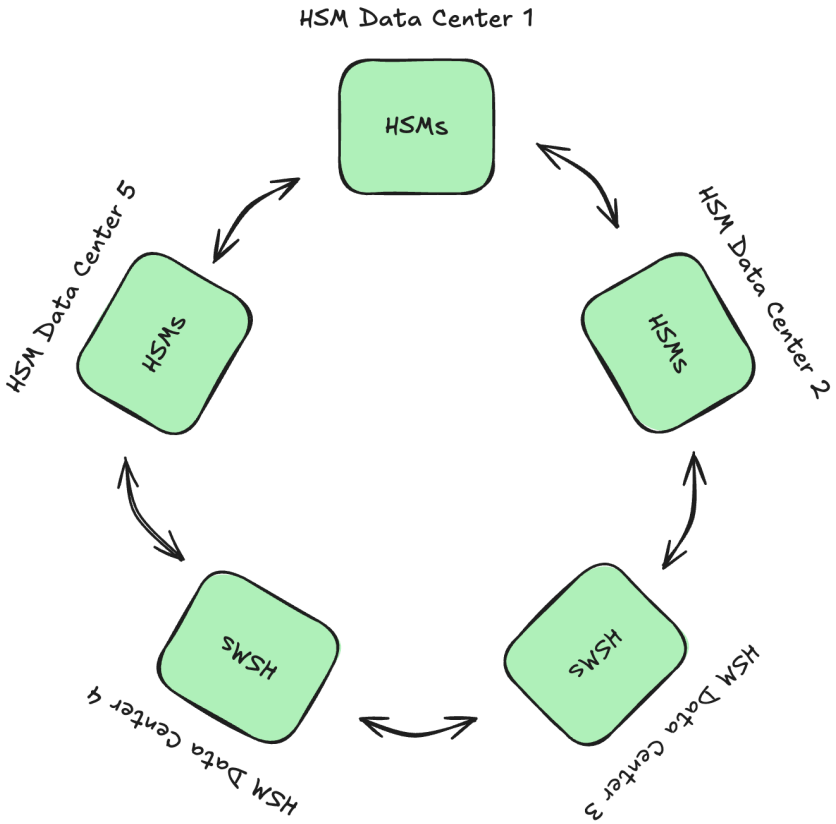
The backups themselves are generated on the client as data files which are encrypted using symmetric encryption with the locally generated key. After a backup is encrypted, it is stored in the third party storage (for example iCloud or Google Drive). Because the backups are encrypted with a key not known to Google or Apple, the cloud provider is incapable of reading them.

HSM-based Backup Key Vault resilience

HSM-based Backup Key Vault is designed to be a majority-consensus based system. That means that its data is replicated among a group of replicas and a simple majority of the replicas is sufficient to be online and to continue system operation. In other words, some replicas can fail, yet the failure is tolerable.

In order to tolerate two kinds of failures (for example when one member replica completely fails and one data center is temporarily disconnected due to a disaster) the HSM-based Backup Key Vault is deployed in seven or more datacenter sites.

HSM-based Backup Key Vault is organized as a fleet or a collection of machines. The fleet is composed of islands or a set of identical replicas. Each island holds a portion of the data that the fleet is responsible for.



Security of client connection to WhatsApp front-end service

Client communication to WhatsApp front-end service for all general needs (including end-to-end encrypted backups) is layered within an encrypted channel. Clients use Noise Pipes from the Noise Protocol Framework for long-running interactive connections. For more information on transport security, see the WhatsApp Encryption Overview whitepaper.

The encrypted backups feature takes advantage of this secure channel to establish trusted connectivity to the server and given that secure channel, in the rest of this whitepaper, we focus only on attacks mounted from within WhatsApp's infrastructure.

Backup generation and backup key registration

Once the end-to-end encrypted backups feature is enabled in the WhatsApp application, the client uses a built-in cryptographically secure pseudorandom number generator to generate a unique encryption key K. The cryptographic strength of K in terms of length is 256 bits, i.e. 32 bytes. That key K is only known to the client and is never transmitted outside of the client unencrypted.

To decrypt the backup, the key K is needed. Thus, to safeguard K in the HSM-based Backup Key Vault, the client performs a registration of K with WhatsApp.

The registration process creates a record within the HSM-based Backup Key Vault associated with the particular client for the purpose of storing the backup encryption key K.

At the heart of the key K registration and an ability to later securely retrieve K is the OPAQUE protocol. The OPAQUE protocol allows the client and the HSM-based Backup Key Vault to establish a registration record inside the HSM-based Backup Key Vault. The OPAQUE protocol allows the record to be secured with a password, without disclosing the actual password to the HSM-based Backup Key Vault.

This record can also later be decrypted to retrieve K for the client, provided that the client is able to produce evidence that it knows the password. This scheme ensures that only the user who knows the password is able to decrypt the record and access the key K.

HSM-based Backup Key Vault validation

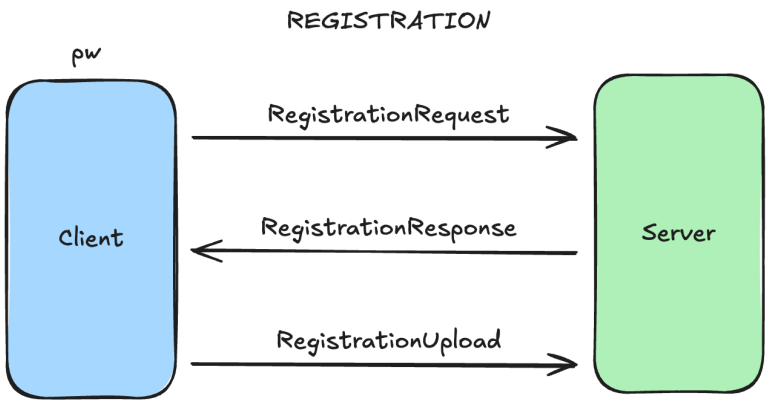
The first step the client performs when interacting with the HSM-based Backup Key Vault is Vault validation. Key Vault clients hardcode the public keys of all supported HSM-based Backup Key Vault fleets. When a client performs an HSM request, the response contains island-specific public keys, accompanied by corresponding cryptographic signatures. The client ensures that the signatures were signed by one of the hardcoded keys before continuing the session with the island-specific keys. This prevents man-in-the-middle attacks.

Registration

The input to the registration process is a user password, that the user has to remember.

The registration process steps are:

1. WhatsApp application asks the user to enter the password.
2. The client utilizes the Client Registration Start function of the `OPAQUE` library to obtain a byte buffer `RegistrationRequest` and sends it to the server. The response to `RegistrationRequest` will be a message `RegistrationResponse`, a signature over `RegistrationResponse` by the HSM `RegistrationResponse_sig`, and a challenge `OPAQUE_C`. The challenge is a random number generated by the server to prevent replay attacks.
3. The client calls the Client Registration Finish `OPAQUE` API, the result of which is a buffer RegistrationUpload and `OPAQUE_K`, a symmetric key which is used to secure the backup key `K`. The client encrypts key `K` using AES-GCM-128 with the first 16 bytes of `OPAQUE_K` (a 64-byte payload) as the key, and sends the resulting ciphertext along with the buffer `RegistrationUpload` to the server.
4. The registration is complete.



Key retrieval

For key retrieval and decryption, the client performs an OPAQUE login operation to derive the same symmetric key on both the client and the server.

Then the client authenticates to the HSM-based Backup Key Vault via OPAQUE login process:

1. The client calls the Client Login Start OPAQUE function, provides the password (pw in the diagram), and obtains a buffer `CredentialRequest`. The client sends `CredentialRequest` to the server, which in turn decrements the attempt count and sends back the buffer `CredentialResponse` and HSM signature `CredentialResponse_sig`.

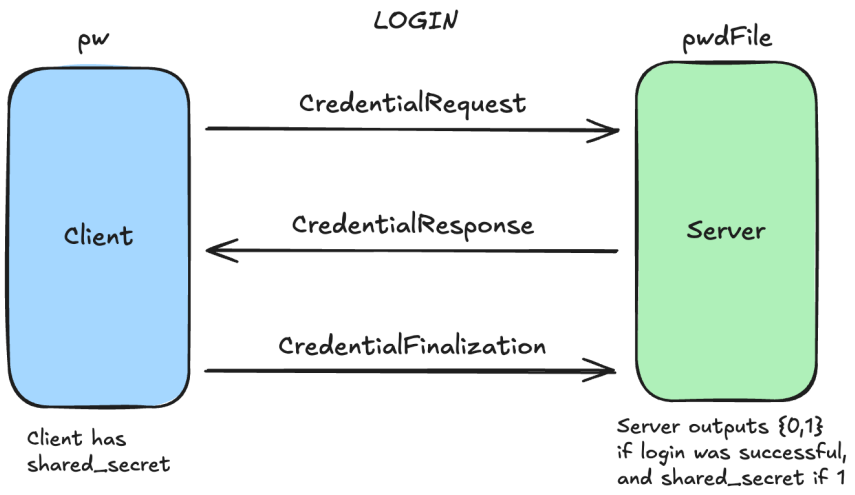
2. The client calls the Client Login Finish OPAQUE function and supplies `CredentialResponse`. The call returns a shared symmetric key `SHARED_K`, symmetric key `OPAQUE_K`, and a buffer `CredentialFinalization`.

3. The client sends `CredentialFinalization` to the server, which returns `EK` and `K_NONCE`. Both are encrypted with `SHARED_K` and `RK_NONCE`, which is a nonce for unwrapping them. `EK` is the encrypted backup key.

4. The client then unwraps `EK || K_NONCE` using `AES-GCM`.

5. Finally, the client unwraps `EK` with `OPAQUE_K` and to get `K: K = AES-GCM(data = EK, key = OPAQUE_K, nonce = K_NONCE)`.

6. Now the client has `K` and can decrypt the backup with it.



Usage Outside of WhatsApp

Over-the-air HSM-based Backup Key Vault Validation

Facebook Messenger leverages this system to store users' recovery codes. To enable older app versions to connect to new HSM clusters, we've built a mechanism to distribute the HSM-based Backup Key Vault fleet public keys over-the-air alongside the HSM response, instead of hardcoding those into the app. The keys are distributed as a Validation Bundle, detailed below:

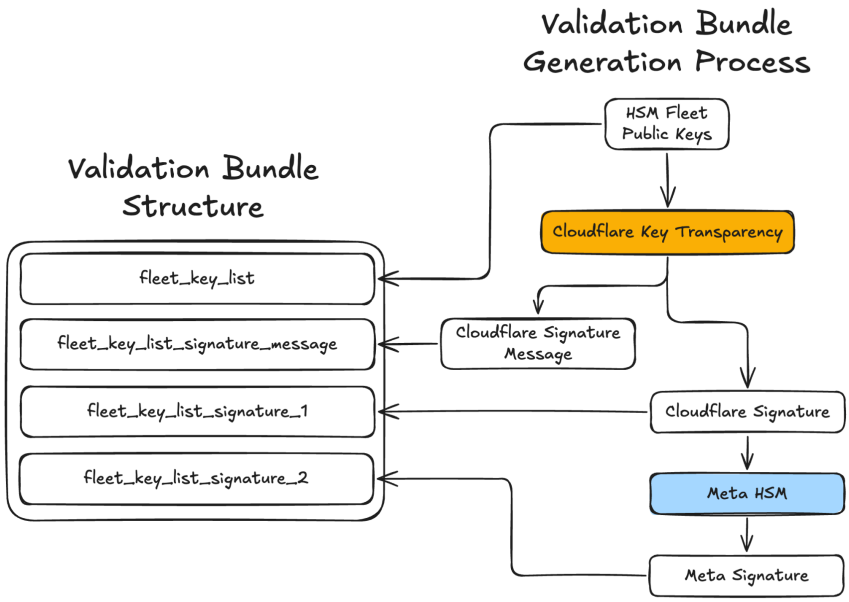
Client Interaction with Fleet Key Lists

When a client initiates a registration or login operation with the HSM-based Backup Key Vault, the server includes the current Validation Bundle in its response. Before proceeding, the client validates the bundle as follows: it confirms that the cryptographic digest of the fleet key list matches the digest in the signed message, then verifies the Cloudflare signature on the bundle's signature message using a baked-in Cloudflare public key, followed by Meta's counter-signature using a baked-in Meta public key. It verifies that the signature message specifies the expected cryptographic algorithm and namespace. It then checks that the bundle's timestamp is within the validity window to ensure freshness. Finally, the client verifies that the responding HSM's public key is present in the validated fleet key list and holds the appropriate role for the requested operation.

Validation Bundle Generation and Structure

Meta periodically generates new Validation Bundles by sending the HSM Fleet Public Keys (structure detailed below) to the Cloudflare Key Transparency service (with the namespace `meta.encrypted_backups.fleet_keys.v1`).

Cloudflare maintains an audit log for every entry, and generates signature artifacts included in the Validation Bundle. Meta signs Cloudflare's signature as an additional signing party. The Validation Bundle is sent to the client upon initialization of HSM connection.



HSM Fleet Public Keys Structure

Keys are delivered as a JSON with the following fields:

```
{
  "keys": [
    {
      "key": HSM_FLEET_PUBLIC_KEY_PLACEHOLDER,
      "roles": [1, 2]
    }, ...
  ],
  "version": 1,
  "padding": INTEGER_PLACEHOLDER,
}
```

Where:

- “keys” contains the list of trusted HSM fleet public keys, with trusted roles (1 = Login, 2 = Register). Each key's role defines which HSM operations that key is authorized to perform: the Login role (1) authorizes the fleet to process key retrieval requests, while the Register role (2) authorizes the fleet to process backup key registration requests.
- "version" indicates the format version of the fleet key list structure, currently set to 1.

- “padding” is used to guarantee unique digests for signature calculation and should be ignored during validation.

Auditing

In order to prove the integrity of the HSM fleet corresponding to the public key, Meta will publish evidence of the secure deployment of each new fleet in its Security & Privacy engineering blog.

Meta provides the `mbt` (Meta Binary Transparency) open-source CLI tool to enable external auditors to verify the consistency of HSM Fleet public keys. The tool is available on Github here. Cloudflare's Ed25519 public key is embedded in the CLI binary as a trust anchor.

To verify a Validation Bundle, run the following command:

```
$ mbt eb-verify-namespace
```

This single command runs the full end-to-end verification and key change analysis pipeline for the encrypted backup fleet keys namespace (`meta.encrypted_backups.fleet_keys.v1`). By default, it verifies the 50 most recent epochs. For each epoch, it performs 7 verification steps:

1. Fetch proof details — retrieves the signature, serialized message, artifact ID, and Meta counter-signature from Meta's Stefi API
2. Download artifact — downloads the artifact binary (HSM Fleet public key list) via CDN to a temporary file
3. SHA-256 digest verification — computes the digest of the downloaded file and confirms it matches the digest recorded in Meta's proof
4. Cloudflare Ed25519 signature verification — verifies the Ed25519 signature over the serialized message using the embedded Cloudflare public key, and confirms the digest inside the serialized message matches the artifact digest
5. Meta counter-signature verification — verifies Meta's HSM signature on Cloudflare's signature bytes using the embedded Meta public key, providing an independent attestation that the proof was seen and validated by Meta's infrastructure
6. Cloudflare AKD auditor cross-check — fetches the independent record from Cloudflare's Key Transparency auditor and compares the digest, signature, and serialized message to Meta's proof
7. Temporary file cleanup — deletes the downloaded artifact regardless of pass/fail

Output on passing:

```
Verifying epoch 100 (1/50)... ✓
Verifying epoch 99 (2/50)... ✓
Verifying epoch 98 (3/50)... ✓
...
Verifying epoch 51 (50/50)... ✓

✓ All 50 proofs verified for
meta.encrypted_backups.fleet_keys.v1

Fleet Keys (epochs 51 → 100):

Epoch  Change    Roles                       Key
─────  ──────    ─────                ──────────
51     baseline  Login, Register           MIIBIjANBgkq...
51     baseline  Login                      MIICIjANBgkq...
51     baseline  Register                   MIIDIjANBgkq...
62     +         Register                   MIIEIjANBgkq...
75     ~         Login, Register → Login    MIIBIjANBgkq...
88     -         Login                      MIICIjANBgkq...
```

Output on failure:

```
Verifying epoch 100 (1/50)... ✓
Verifying epoch 99 (2/50)... ✓
Verifying epoch 98 (3/50)... ✗
Verifying epoch 97 (4/50)... ✓
...
Verifying epoch 52 (49/50)... ⚠
Verifying epoch 51 (50/50)... ✓

Verified 50 proofs: 47 ✓  2 ✗  1 ⚠

✗ 47/50 proofs verified for
meta.encrypted_backups.fleet_keys.v1

Epoch  Step
─────  ────────────────────
98     Digest verification
73     Cloudflare signature
52     Artifact download
```

```
Fleet Keys (epochs 51 → 100):

Epoch  Change    Roles                        Key
─────  ──────    ─────                ──────
51     baseline  Login, Register           MIIBIjANBgkq...
51     baseline  Login                      MIICIjANBgkq...
62     +         Register                   MIIEIjANBgkq...
75     ~         Login, Register → Login    MIIBIjANBgkq...
88     -         Login                      MIICIjANBgkq...
```

If verification passes, the auditor has confirmed that:

1. The HSM Fleet public key list artifacts have not been tampered with (digest verification)
2. Cloudflare has cryptographically signed each artifact's digest (signature verification)
3. Meta has counter-signed Cloudflare's signatures (counter-signature verification)
4. Meta's records are consistent with Cloudflare's independent audit log (cross-check)

This confirms that the HSM Fleet public keys distributed to clients are authentic and have been independently verified by Cloudflare's Key Transparency service. For advanced users, the individual verification steps are also available as separate commands (`mbt proofs`, `mbt proof`, `mbt verify-digest`, `mbt verify-signature`, `mbt audit`) for inspecting specific epochs or debugging individual verification failures.

Conclusion

In this document we discussed the mechanisms that are employed in WhatsApp's end-to-end encrypted backup solution to help keep users' backups secure.

Our goal with this project is to create a system to provide additional security for users' message history from WhatsApp, as well as the users' own cloud providers and any other third parties.

WhatsApp is committed to building technologies that safeguard the content of users' messages which includes launching end-to-end encryption at scale in

2016 and now advancing it further to protect messages at rest in the cloud with end-to-end encrypted backups.

Resources

1. OPAQUE publication: https://eprint.iacr.org/2018/163.pdf
2. The OPAQUE Augmented Password-Authenticated Key Exchange (aPAKE) Protocol (RFC 9807): https://datatracker.ietf.org/doc/rfc9807/
3. The OPAQUE key exchange protocol implementation in Rust: https://github.com/facebook/opaque-ke
4. OPAQUE documentation https://docs.rs/opaque-ke/